PYTHON – ESTRUTURA DA DADOS

LISTAS...

São coleções (caracteres, strings, números...) ordenados, separados por vírgula e que estão dentro de colchetes (limitados por colchetes).

- Exemplo de Listas HOMOGÊNEAS:

  [1, 2, 3, 4,.....]
  ['a', 'b', 'c',.....]
  ['texto1', 'texto2',.....]

- Exemplo de Listas HETEROGÊNEAS:

  [1, 'a', 'texto1', [1, 2, 3], 311,.....]

LISTAS...

- ✓ Atribuição e tipo
- ✓ Operações básicas
- ✓ A função list()
- ✓ Indexação e fatiamento de listas
- ✓ Modificação de itens de uma lista

LISTAS...

Funções builtins usadas em Listas... (integradas)

FUNÇÕES	DESCRIÇÃO
`len( )`	Retorna o número de elementos de uma lista.
`sum( )`	Retorna a soma dos números de uma lista.
`any( )`	Retorna True se qualquer um dos valores booleanos de uma lista for verdadeiro.
`all( )`	Retorna True se todos os valores booleanos de uma lista forem verdadeiros.
`sorted( )`	Retorna uma cópia modificada(ordenada) de uma listas e mantem a lista original intacta.

LISTAS...

Métodos de uma Lista...

FUNÇÕES	SINTAXE	DESCRIÇÃO
append( )	List.append(item)	Insere um elemento no final da lista. Modifica a original.
count( )	List.count(item)	Retorna a quantidade de vezes que um item ocorre em uma lista.
insert( )	List.insert(index, item)	Insere um item em um dado local (index), deslocando os elementos da lista para direita.
extend( )	List.extend(List2)	Adiciona os itens da Lista2 no final da lista List.
index( )	List.index(item)	Busca um item em uma lista e retorna a sua posição. Se tiver mais de uma ocorrência do item, retorna a primeira. Se não encontrar o item, retorna Erro para o método.
remove( )	List.remove(item)	Remove a primeira ocorrência do item em uma lista. Se o item não estiver presente na lista, retorna um erro para o método.
sort( )	List.sort( )	Ordena os itens de uma lista(na própria lista). Modifica a original.
reverse( )	List.reverse( )	Reverte a ordem dos itens de uma lista(na própria lista). Modifica a lista original diretamente.
pop( )	List.pop([index])	Remove um determinado item de uma lista (retornando o valor removido). Se o valor de index for omitido, retorna o valor mais a direita.

***"List" deve ser substituído pelo nome atual da lista (nome da variável)

Testes em terminal...

Em Python, tanto métodos quanto funções são blocos de código nomeados que realizam tarefas, mas a principal diferença reside em onde e como são chamados. Funções são entidades independentes, chamadas diretamente pelo nome, enquanto métodos são funções associadas a um objeto (ou classe) e são chamados usando a notação de ponto.

Funções:
São blocos de código que podem ser definidos e chamados em qualquer lugar do programa. Não estão intrinsecamente ligadas a nenhum objeto específico.
Exemplos: print(), len(), range().

Métodos:
São funções associadas a um objeto ou classe. São chamados usando a notação de ponto, por exemplo, **objeto.metodo().**
Operam em dados ou estado dentro do objeto ao qual estão ligados.
Exemplos: minha_string.upper(), meu_dicionario.get(), meu_objeto.metodo().

Em resumo:

Tanto métodos quanto funções são blocos de código reutilizáveis.

Funções são independentes e globais, enquanto métodos são específicos de um objeto ou classe.

Métodos são usados para manipular o estado e dados dentro de um objeto, enquanto funções podem ser usadas para uma variedade de propósitos, incluindo manipulação de dados, cálculos, e interação com o usuário.

Built-ins são funções nativas que já vêm prontas com o Python, ficam sempre disponíveis e não exigem nenhum ponto de acesso. Já os **métodos** são funções que pertencem a um objeto específico e só podem ser chamadas a partir dele.

LISTAS...

- ✓ **Fazer um programa em Python que:**
  - ❑ **Receba uma quantidade ilimitada (não se sabe quantas) de nome de cidades e guarde essas cidades em uma lista.**
  - ❑ **Para encerrar a digitação das cidades, deve-se digitar: sair.**
  - ❑ **Em seguida, deve-se ordenar essa lista de cidades em ordem crescente.**
  - ❑ **Mostrar a relação de cidades ordenadas, uma por linha.**

TUPLAS...

São coleções (caracteres, strings, números...) ordenados, separados por vírgula e que estão dentro de parênteses (limitados por parênteses). São coleções IMUTÁVEIS.

- Exemplo de Tuplas HOMOGÊNEAS:

  (1, 2, 3, 4,.....)
  ('a', 'b', 'c',.....)
  ('texto1', 'texto2',.....)

- Exemplo de Tuplas HETEROGÊNEAS:

  (1, 'a', 'texto1', (1, 2, 3), 311,.....)

TUPLAS...

É indicado usar tuplas em Python quando você precisa armazenar uma coleção de elementos em uma ordem específica e tem a certeza de que esses dados não devem ser alterados ao longo do programa. Por serem imutáveis, elas garantem maior segurança ao código, além de serem mais rápidas e consumirem menos memória.

Casos de uso ideais para Tuplas

- Dados protegidos: Para guardar informações fixas e pessoais, como o número de um CPF, data de nascimento ou coordenadas geográficas.
- Retorno de funções: Quando uma função precisa devolver mais de um valor ao mesmo tempo.

- **Chaves de dicionário: Como tuplas são imutáveis, elas podem ser usadas como chaves em dicionários ou itens em conjuntos (sets), o que não é permitido com listas.**
- **Agrupamento lógico: Para manter juntos dados que formam um único registro (ex: um nome, seguido de sua idade e altura).**
- **Principais diferenças para as Listas**
- **Para entender quando usar qual, considere esta regra prática:**

Característica	Tupla ()	Lista []
Imutável (não pode mudar)	Sim	Não
Mais rápida	Sim	Não
Uso de memória	Baixo	Mais alto

TUPLAS...

- ✓ Atribuição e tipo
- ✓ Operações básicas
- ✓ A função tuple()
- ✓ Indexação e fatiamento de Tuplas

TUPLAS...

Funções builtins usadas em Tuplas... (integradas)

FUNÇÕES	DESCRIÇÃO
`len( )`	Retorna o número de elementos de uma tupla.
`sum( )`	Retorna a soma dos números de uma tupla.
`sorted( )`	Retorna uma cópia modificada(ordenada) de uma tupla e mantem a tupla original intacta.

TUPLAS...

Métodos de uma Tupla...

FUNÇÕES	SINTAXE	DESCRIÇÃO
count( )	Tupla.count(item)	Retorna a quantidade de vezes que um item ocorre em uma tupla.
index( )	Tupla.index(item)	Busca um item em uma tupla e retorna a sua posição. Se tiver mais de uma ocorrência do item, retorna a primeira. Se não encontrar o item, retorna Erro para o método.

***"Tupla" deve ser substituído pelo nome atual da tupla (nome da variável)

Testes em terminal...

TUPLAS...

- ✓ **Fazer um programa em Python que:**

  - ❑ **Receba uma quantidade ilimitada (não se sabe quantas) de idade de pessoas.**
  - ❑ **Guarde essas idades em uma lista.**
  - ❑ **Para encerrar a digitação das idades, deve-se digitar: -1.**
  - ❑ **Em seguida, deve-se gerar uma tupla de idades, a partir da lista.**
  - ❑ **Deve-se mostrar as seguinte informações, quando consultado a tupla:**
    - **a) Qtd de idades digitadas.**
    - **b) Média das idade.**

DICIONÁRIOS...

- ✓ São coleções de um conjunto de pares chave:valor, com uma restrição de que a chave seja única dentro da coleção.
- ✓ Os dicionários são construídos, utilizando chaves { }
- ✓ Utiliza-se ":" para separar a chave do valor.
- ✓ Sempre o operador a esquerda é a chave e o a direita, o valor.
- ✓ Os dicionários são indexados pela chave.
- ✓ São as chaves "case sensitive"

DICIONÁRIOS...

- Exemplo de Dicionários:
  peixes = {“t”:”tubarão”, “b”:”baleia”, “a”:”atum”}
  f1 = {1:”Mercedes”, 2:”Ferrari”, 3:”Renault”}

- Exemplo de Dicionários (heterogêneo):
  dic_misto = {1:45, ‘a’:34.58, ‘teste’:’frase’, 3:’nome’}

DICIONÁRIOS...

Os dicionários em Python são indicados quando você precisa armazenar dados em pares de chave e valor, onde cada valor é acessado por um nome ou identificador único, e não por uma posição numérica. Eles são ideais para buscas rápidas e para representar dados do mundo real.

DICIONÁRIOS...

- **Associação de dados: Quando você precisa vincular uma informação à outra, como o nome de um produto e o seu preço, ou o CPF de uma pessoa e os seus dados cadastrais.**
- **Buscas muito rápidas: Se você precisa encontrar um item específico frequentemente, os dicionários são muito mais rápidos do que procurar em uma lista.**
- **Evitar itens duplicados: As chaves de um dicionário precisam ser únicas, o que ajuda a garantir que não haverá repetição de identificadores (como dois usuários com o mesmo e-mail).**
- **Representação de registros complexos: São ótimos para guardar os detalhes de um único objeto (ex: { 'nome': 'Ana', 'idade': 25, 'ativo': True }), o que se assemelha muito a formatos de dados como o JSON.**

DICIONÁRIOS...

- ✓ Atribuição e tipo
- ✓ Operações básicas
- ✓ A função dict()
- ✓ Indexação e Alteração

DICIONÁRIOS...

Funções builtins usadas em dicionários... (integradas)

FUNÇÕES	DESCRIÇÃO
len( )	Retorna o número de duplas (chave:valor) de um dicionário.
all( )	Retorna True se todos os valores das chaves do dicionário forem verdadeiras. Caso contrário, FALSE.
any( )	Retorna True se pelo menos um valor das chaves do dicionário for verdadeiro. Caso contrário, FALSE.
sorted( )	Retorna uma **lista** modificada(ordenada) de um dicionário baseado em suas chaves. Mantem intacto o dicionário.

DICIONÁRIOS...

Métodos de um Dicionário...

FUNÇÕES	SINTAXE	DESCRIÇÃO
clear( )	Dic.clear( )	Remove todos os pares chave:valor de um dicionário.
fromkeys( )	Dic.fromkeys(seq[,valor])	Cria um novo dicionário a partir da sequencia dos elementos com um valor informado.
get( )	Dic.get(chave[,padrão])	Retorna o valor associado à chave especificada no dicionário. Se a chave não estiver presente, ela retornará o valor padrão. Se padrão não for fornecido, o padrão será None, para que esse método nunca gere um KeyError.
items( )	Dic.items( )	Retorna uma nova visão dos pares de chave e valor do dicionário como tuplas.
keys( )	Dic.keys( )	Retorna uma nova view que consiste em todas as chaves do dicionário.
pop( )	Dic.pop(chave[,padrão])	Remove a chave do dicionário e retorna seu valor. Se a chave não estiver presente, ela retornará o valor padrão. Se o padrão não for fornecido e a chave não estiver no dicionário, isso resultará em KeyError.

***“Dic” deve ser substituído pelo nome atual da lista (nome da variável)

DICIONÁRIOS...

Métodos de um Dicionário...

FUNÇÕES	SINTAXE	DESCRIÇÃO
popitem( )	Dic.popitem( )	Remove e retorna um par chave-valor arbitrário (como uma tupla) de um dicionário. Se o dicionário estiver vazio, a chamada a popitem() causará um erro KeyError.
setdefault( )	Dic.setdefault(chave[,padrão])	Retornar o valor associado a uma chave em um dicionário. Se a chave estiver presente, ele retorna o valor correspondente. Se a chave não estiver no dicionário, ela é inserida com um valor padrão e esse valor padrão é retornado. O valor padrão é None por padrão, mas você pode fornecer um valor diferente como argumento.
update( )	Dic.update([outro])	Atualiza o dicionário com os pares chave:valor de outro objeto de dicionário.
values( )	Dic.values( )	Retorna uma nova view que consiste em todos os valores do dicionário.

***“Dic” deve ser substituído pelo nome atual da lista (nome da variável)

Testes em terminal...

DICIONÁRIOS...

- ✓ **Fazer um programa em Python que:**

  - ❑ **Receba o cadastro de uma qtd ilimitada (não se sabe quantas) e proprietários e seus respectivos apartamentos (número do apto e nome do proprietário).**
  - ❑ **Guarde essas informações em um dicionário, onde a chave de busca é o número do apto.**
  - ❑ **Para encerrar a digitação o número do apto será -1.**
  - ❑ **Em seguida, deve-se mostrar uma listagem em ordem crescente de apto: apto – nome do proprietário.**
  - ❑ **No final apresente a quantidade total de apto listados.**

CONJUNTOS (SETS)...

- ✓ São coleções não ordenadas (sem índice) que não permitem a duplicação de elementos.
- ✓ Os conjuntos são delimitados, utilizando chaves{ }
- ✓ Os conjuntos suportam as operações matemáticas de união, intersecção, diferença e diferença simétrica.
- ✓ São estruturas MUTÁVEIS.

❑ Exemplo de Conjuntos (sets):

peixes = {“tubarão”, “baleia”, “atum”, “sardinha”}
numeros = {1, 2, 3, 6, 9, 23, 67}
mix = {1, ‘a’, ‘nome’, 3.58, (2, 4, 5)}

Conjuntos (sets) em Python são ideais quando você precisa eliminar valores duplicados de uma coleção e realizar buscas rápidas para verificar se um item existe. Eles são otimizados para operações matemáticas como uniões e interseções.

1. **Remoção de duplicatas**

Se você tem uma lista com itens repetidos, convertê-la para um conjunto remove automaticamente todas as repetições.

Exemplo: Converter `[1, 1, 2, 3]` para `{1, 2, 3}`.

2. **Verificação rápida de existência (lookup)**

Saber se um elemento está presente em um conjunto é muito mais rápido do que procurar em uma lista, especialmente com grandes volumes de dados.

Uso: Checar se um usuário já está na lista de banidos ou se um item está no carrinho de compras.

3. **Operações matemáticas de conjuntos**

Úteis para cruzar dados de duas ou mais coleções diferentes.

Interseção: Encontrar elementos que estão em ambos os conjuntos (ex: produtos que estão em promoção e no seu carrinho).

União: Juntar todos os elementos de dois conjuntos (ex: lista de e-mails da lista A ou da lista B).

Diferença: Encontrar elementos presentes em um conjunto, mas não no outro.

CONJUNTOS (SETS)...

- ✓ Atribuição e tipo
- ✓ Operações básicas
- ✓ A função set()

CONJUNTOS (SETS)...

Funções builtins usadas em Conjuntos... (integradas)

FUNÇÕES	DESCRIÇÃO
`len( )`	Retorna o número de elementos de um conjunto.
`sorted( )`	Retorna uma **lista** modificada(ordenada) de um conjunto(set). Mantem intacto o conjunto original.

CONJUNTOS (SETS)...

Métodos de um Conjunto(set)...

FUNÇÕES	SINTAXE	DESCRIÇÃO
add( )	Set.add(item)	Adiciona um item ao conjunto Set.
clear( )	Set.clear( )	Remove todos os itens do conjunto Set.
diference( )	Set.diference(*outros)	Retorna um novo conjunto com itens no conjunto Set que não estão nos outros conjuntos.
discard( )	Set.discard(item)	Remove um item do conjunto Set se estiver presente.
intersection( )	Set.intersection(*outros)	Retorna um novo conjunto com itens comuns ao conjunto Set e todos os outros conjuntos.
indisjoint( )	Set.isdisjoint(outro)	Retorna True se e somente se dois conjuntos não tiverem nenhum elemento em comum. Em outras palavras, a interseção dos dois conjuntos é o conjunto vazio.
issubset( )	Set.issubset(outro)	Retorna True se todos os itens de um conjunto (o conjunto que chama o método) estiverem presentes em outro conjunto que é passado como argumento. Em outras palavras, ele verifica se o conjunto que chama é um subconjunto do outro conjunto. Se for, retorna True. Caso contrário, retorna False.

***“Set” deve ser substituído pelo nome atual da lista (nome da variável)

CONJUNTOS (SETS)...

Métodos de um Conjunto(set)...

FUNÇÕES	SINTAXE	DESCRIÇÃO
isuperset( )	Set.isuperset(outro)	Retorna True se cada elemento em outro conjunto estiver no conjunto Set.
pop( )	Set.pop( )	Remove e retorna um item arbitrário do conjunto Set. Ele gera KeyError se o conjunto for vazio.
remove( )	Set.remove(item)	Remove um item do conjunto Ser. Ele gera KeyError se o item não estiver contido no conjunto.
symmetric_diference( )	Set.symetric_diference(outro)	Retorna um novo conjunto com itens no Conjunto Set ou no conjunto outro, mas não em ambos.((ou exclusivo))
union( )	Set.union(*outros)	Combina os elementos de diferentes conjuntos, criando um novo conjunto com todos os elementos únicos presentes em todos os conjuntos originais.
update( )	Set.update(*outro)	Atualiza o conjunto adicionando elementos de outro iterável (como outro conjunto, lista ou tupla). Ele garante que apenas elementos únicos sejam adicionados, evitando duplicatas.

***"Set" deve ser substituído pelo nome atual da lista (nome da variável)

Testes em terminal...

CONJUNTOS (SETS)...

- ✓ **Fazer um programa em Python que:**

  - ❑ **Receba uma quantidade de variável (não definida) de strings (termine a digitação com string vazia = " ")**
  - ❑ **Guarde essas informações em uma lista.**
  - ❑ **Crie uma outra lista onde cada elemento (palavra) se transformará em uma lista de caracteres únicos, ((caracteres não repetidos)). Por exemplo:**
  - ❑ **Se uma palavra de lista for igual a: 'barbante', você a modificará para ['b', 'a', 'r', 'n', 't', 'e'], tirando o que tem repetido.**
  - ❑ **Em seguida, mostre a lista modificada.**

- ✓ **Fazer um programa em Python que:**

- **Receba as informações (nome, idade e n. do calçado) de 5 pessoas distintas.**
- **Depois de receber as informações você deve mostrar a relação completa das pessoas e suas informações, em ordem alfabética.**
- **Ao final do relatório, você deve calcular e mostrar as seguintes informações:**
- **A média de idade das pessoas**
- **Número médio de calçado das pessoas pesquisadas.**

MATTHES, E. *Curso Intensivo de Python*: Uma Introdução Prática e Baseada em Projetos à Programação. São Paulo: Novatec, 2016
MENEZES, N. N. C. *Introdução à Programação com Python*: Algoritmos e Lógica de Programação. São Paulo: Novatec, 2019.
CRUZ, F. *Python*: escreva seus primeiros programas. São Paulo: Casa do Código, 2016.
Python Profissional: do zero ao mercado de trabalho! - Dilermando Piva Junior Professor | Autor | Data Scientist.
Python Completo do Zero ao Avançado + Projetos Reais 2025 - Andre Iacono AI · Cloud · Developer |Educating with Passion, Transforming with Purpose. Programação em Python do básico ao avançado - Geek University.